EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Project: Patient Appointment Scheduling System | Platform: Zoho Creator

Saranya V. | Register No: 2116241501188 | AIML | Rajalakshmi Engineering College

Abstract

This case study examines the limitations that may arise when a low-code application is expanded into a complex, multi-branch healthcare appointment platform. The Patient Appointment Scheduling System built in Zoho Creator currently contains an appointment form, report, dashboard components and a Deluge workflow for calculating net payable fees. These components are suitable for workflow-centred tasks, but enterprise-scale requirements such as high concurrency, real-time availability, complex scheduling, external integrations, large data volumes and strict governance may require additional services.

1. Introduction

Low-code platforms allow applications to be built quickly using forms, reports, visual pages and workflows. They are useful for internal business processes and small-to-medium applications. However, an application that works well for one clinic may need a different architecture when extended to a large hospital network with many branches, thousands of simultaneous users and integrations with external healthcare systems.

2. Case Scenario

The following scenario is fictional and created for academic analysis. A healthcare group begins with the current Patient Appointment Scheduling System in Zoho Creator at one clinic. Management later proposes to use it across 100 clinics, with live doctor availability, simultaneous appointment requests, automated fee calculation, patient notifications, electronic medical record integration, payment processing and group-wide analytics.

3. Analysis of Limitations

Area	Potential low-code limitation	Impact on the expanded project
Performance and scalability	Workflow execution, API usage and platform resources may have limits.	Busy appointment periods and large reports may become slow or hit execution limits.
Concurrency and booking integrity	Fine-grained control over transaction locking may be limited.	Two users could attempt to book the same doctor/time slot unless availability is checked atomically.
Complex scheduling algorithms	Visual rules and Deluge are best suited to business logic, not heavy optimization.	Multi-clinic scheduling, travel time, urgency and doctor workload optimization may need a separate service.
Integration depth	Connectors and REST APIs cover many common cases but may not fit every legacy interface.	Hospital records, laboratory, pharmacy, payment and insurance systems may need middleware.
Data volume and analytics	Large-scale reporting may be constrained by query and platform limits.	Long-term patient history and cross-clinic analytics may require a data warehouse or BI service.
UI customization	The UI is constrained by supported page-builder components and layouts.	A highly customized patient portal or mobile booking experience may require a pro-code frontend.
Vendor lock-in and portability	Forms, workflows and scripts depend on the platform's runtime and data model.	Migration can be costly if the platform's pricing, features or limits change.
Governance, security and testing	Many apps and users increase permission, audit and release-management complexity.	Inconsistent permissions or weak testing can expose patient data or disrupt bookings.

4. Mitigation Strategies

• Use Zoho Creator for appointment forms, routine workflows, approvals, staff-facing reports and operational dashboards.

• Move high-concurrency slot allocation and complex scheduling logic to a dedicated backend service with transaction-safe booking rules.

• Use an API gateway or integration layer for electronic medical records, payment providers, notifications and legacy systems.

• Move large-scale historical analytics to a data warehouse and BI platform.

• Apply least-privilege permissions, review data-sharing rules, document workflows, and maintain separate Development, Stage and Production environments.

• Run a proof of concept and load tests using realistic peak traffic before scaling to all clinics.

• Keep data exportable and document business rules and API contracts to reduce vendor lock-in.

5. Recommended Architecture

Component	Recommended approach
Appointment form and routine staff workflows	Zoho Creator low-code
Live doctor availability and slot locking	Pro-code service with transaction-safe storage
Fee calculation	Keep simple rules in Deluge; move complex pricing/insurance logic to a service if it grows
Notifications and external records	API integration layer/connectors
Group-wide analytics	External data warehouse and BI tool
Governance	Role-based access, audit review, versioned releases and load testing

6. Project Screenshots and Observations

The supplied screenshots show a working form layout, a report with five sample records, a dashboard design and a Deluge fee-calculation workflow. They provide evidence of the current project's workflow-centred low-code structure. They do not measure performance under heavy load, prove a concurrency

failure, or demonstrate external hospital integrations; those items are analyzed as potential limitations for the fictional expanded scenario.

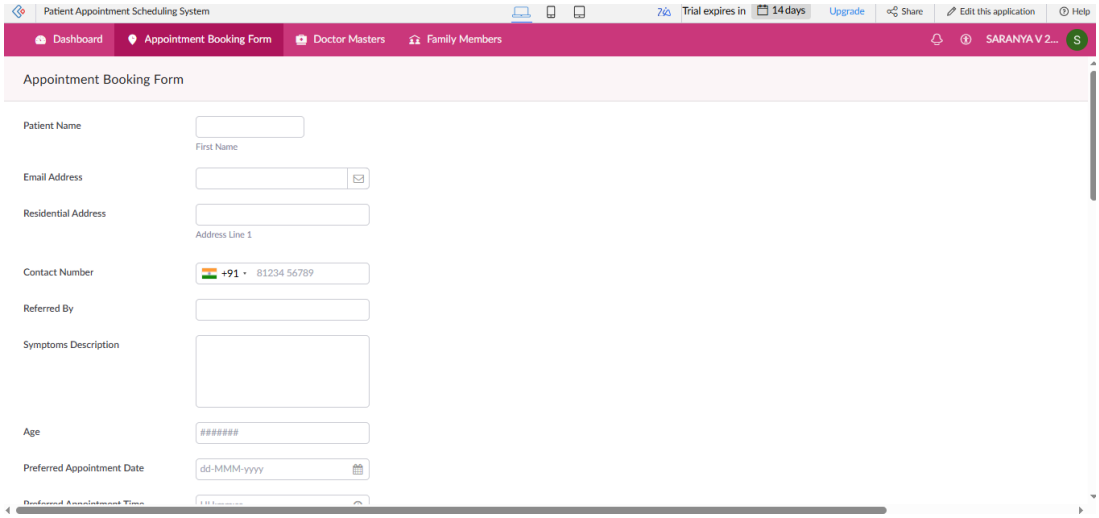


Figure 1: Appointment Booking Form in the Patient Appointment Scheduling System.

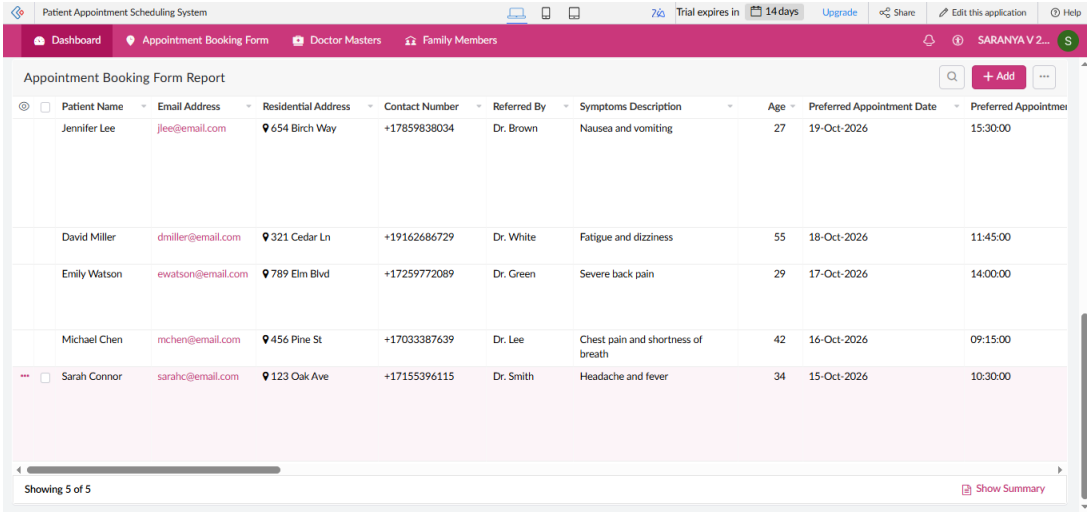


Figure 2: Appointment Booking Form Report showing saved sample appointment records.

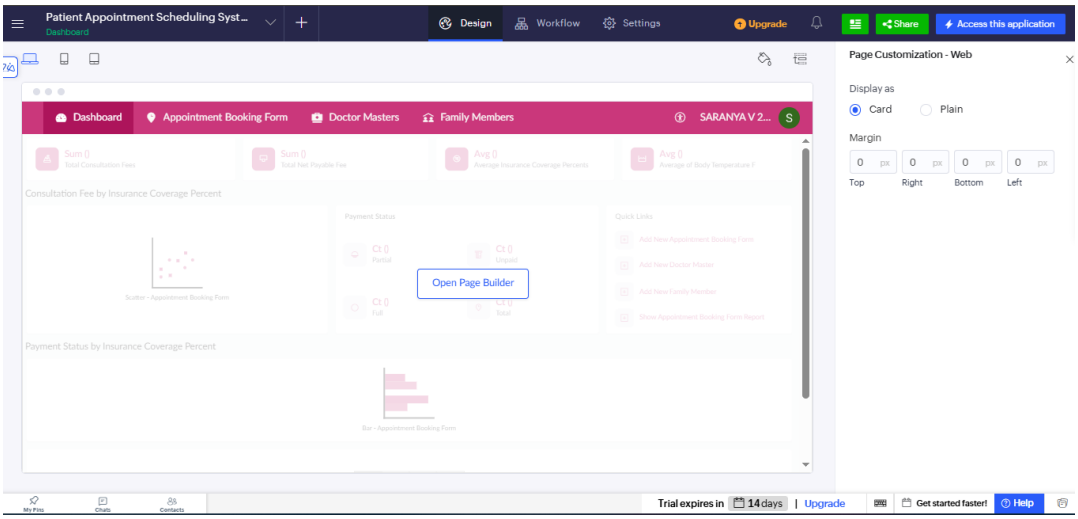


Figure 3: Dashboard design with summary cards, payment-status panel, quick links and charts.

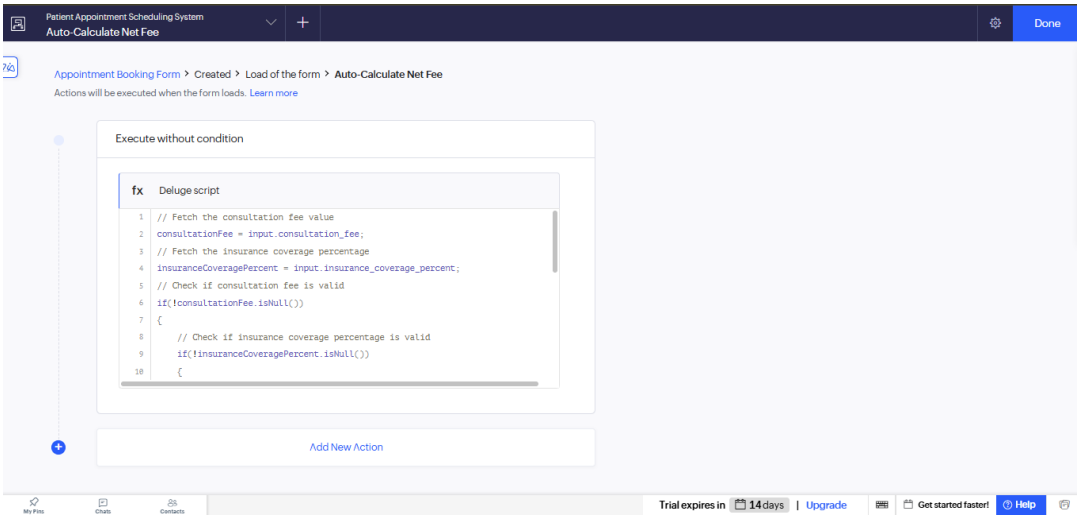


Figure 4: Existing Auto-Calculate Net Fee workflow using Deluge.

7. Recommended Decision

A full replacement of the appointment application with custom code is not necessary. Keep Zoho Creator for forms, routine workflows and staff-facing reporting. For a multi-clinic system, move high-concurrency booking, advanced scheduling and large-scale analytics to specialized services connected through APIs. Confirm the design with a proof of concept, security review and load tests.

8. Conclusion

Low-code platforms are effective for workflow-centred applications, and the current Patient Appointment Scheduling System demonstrates how forms, reports, dashboards and Deluge workflows can be combined quickly. When the same application is expanded to a large healthcare network, concurrency, scalability, complex algorithms, integration depth, data volume, customization, governance and portability become important concerns. A hybrid architecture balances low-code development speed with the control of pro-code services.

9. Learning Outcome

Learnt to evaluate the suitability of low-code platforms for an enterprise use case, identify technical and organizational limitations, and recommend a hybrid architecture that combines Zoho Creator with dedicated backend services and external analytics.